SATELINK NETWORK

Full Repository Audit Report

Production Launch Readiness Assessment

Date: April 1, 2026

Scope: Full-stack system audit (backend, frontend, contracts, infra, security)

Branch: audit/snapshot (main: b6da989)

Verdict: NOT READY for production launch | Overall Score: 3.9 / 10

This report provides a complete, evidence-based assessment of what Satelink has built, what is missing, what is broken, and what must be done to reach production launch with real users and real money. All findings are derived from direct code inspection across all branches, services, and configurations.

TABLE OF CONTENTS

1. System Overview
2. What Is Already Built
3. Missing Product Layers
4. End-to-End Flow Analysis
5. Architecture Audit
6. Database Audit
7. Docker & Infrastructure Audit
8. Codebase Health
9. Security Audit
10. Performance & Scale
11. UI/UX Product Readiness
12. CI/CD & GitHub Readiness
13. Launch Readiness Score
14. Cleanup Plan (Safe Mode)
15. Execution Roadmap

SECTION 1 — SYSTEM OVERVIEW

Satelink is a DePIN (Decentralized Physical Infrastructure Network) platform that routes real-world computational workloads — RPC relaying, AI inference, automation jobs, webhook delivery, and bandwidth sharing — through a decentralized node network. Revenue is settled in USDT on Fuse Network using a 50/30/20 split model (Node Operators / Platform / Distributors).

Architecture Overview

```
CLIENTS (Enterprise, Builder, Public RPC)
| HTTPS / JWT / API Key
v
GATEWAY LAYER (Express.js) — 300+ endpoints, 60+ route files
|
+--> Operations Engine (pricing, ops counting)
+--> Revenue Oracle (event recording, market monitoring)
+--> Settlement Engine (Fuse EVM adapters, deposit detection)
+--> Workload Router (RPC, AI, automation, webhook)
|
v
PostgreSQL (60+ tables) + Redis Streams (5 priority job queues)
|
v
Smart Contracts (Fuse Network) — 9 Solidity files, OpenZeppelin v5.4
|
v
NODE NETWORK — router | edge | vps | managed types

FRONTEND: Next.js 16 Dashboard (React 19, TypeScript 5, Tailwind v4)
100+ routes | 5 role-based portals | SSE real-time streaming
```

Tech Stack

Component	Technology
Backend	Node.js / Express (monorepo via Turbo)
Frontend	Next.js 16.1.6, React 19, TypeScript 5, Tailwind v4, shadcn/ui
Database	PostgreSQL (60+ tables, pg_adapter with SQLite translation layer)
Queue	Redis Streams (5 priority levels: CRITICAL, HIGH, NORMAL, LOW, DEAD)
Blockchain	Fuse Network, Solidity ^0.8.20, OpenZeppelin v5.4, Foundry
Auth	JWT + RBAC (6 role types: admin_super/ops/readonly, node_operator, builder, distributor, enterprise)
Monitoring	Prometheus + Grafana + AlertManager (basic configuration)
CI/CD	GitHub Actions (4 workflows)
Containers	Docker Compose (multi-service)

SECTION 2 — WHAT IS ALREADY BUILT

Backend — Working

[OK] Express server with graceful shutdown, retry logic, module initialization
[OK] PostgreSQL database adapter with connection pooling (max 20 connections)
[OK] JWT authentication with role-based access control (6 role types)
[OK] Epoch aggregation scheduler (60s interval) — atomic 50/30/20 revenue split
[OK] Revenue event recording (revenue_events_v2 table)
[OK] Node registration and lifecycle management
[OK] Admin control room: 55+ endpoints (emergency lockdown, security freeze, diagnostics)
[OK] Admin API V2: 25+ endpoints (stats, treasury, epochs, nodes, withdrawals)
[OK] Auth endpoints (login, register, JWT refresh, wallet-based auth)
[OK] System health + Prometheus metrics endpoints
[OK] Structured Winston logging + Helmet security headers
[OK] Database schema auto-creation with 60+ tables

Backend — Partial

[PARTIAL] Settlement engine — adapter pattern exists, SimulatedAdapter works, EvmAdapter stubbed
[PARTIAL] Redis Streams job queue — enqueue/dequeue works, consumer groups partially wired
[PARTIAL] RPC gateway — relay exists, full routing incomplete
[PARTIAL] Workload connectors — 8 market connectors, ~50% scaffolding
[PARTIAL] Economics engines (pricing, breakeven, profitability) — ~30% real logic
[PARTIAL] Node reputation system — tracking exists, scoring incomplete
[PARTIAL] SLA engine — credit tracking exists, breach calculations partial
[PARTIAL] Enterprise billing — routes exist, full billing flow incomplete
[PARTIAL] Growth engine — referral tracking exists, incentive logic partial
[PARTIAL] Automation scheduler — framework exists, most jobs stubbed

Backend — Broken

[BROKEN] operations_engine.js:311 — **`db` is undefined** (ReferenceError on epoch finalization)
[BROKEN] futures_escrow.js — Multiple missing `await` keywords (returns Promises, not values)
[BROKEN] job_escrow.js:25 — Missing await (data corruption risk)

API Endpoint Summary

Category	Count	Status
Admin Control Room	55+	Working
Admin API V2	25+	Working
Auth & Sessions	15+	Working

Category	Count	Status
System Health	10+	Working
Node Operations	15+	Partial
Distributor API	10+	Partial
Builder API	15+	Partial
Enterprise API	10+	Partial
Workload/Jobs	20+	Partial
Growth & Referrals	10+	Partial
TOTAL	300+	~60% complete

Smart Contracts (9 Solidity Files — All Compile Clean)

Contract	Purpose	Status
NodeRegistryV2.sol	Node registration with REGISTRAR_ROLE, lifecycle management	Production Ready
RevenueVault.sol	USDT custody with WITHDRAWER_ROLE access control	Production Ready
RevenueDistributor.sol	50/30/20 split with SafeERC20, infra reserve	Production Ready
ClaimsContract.sol	Two-phase claim mechanism with expiry	Production Ready
ClaimsWithdrawals.sol	Merkle proof-based epoch claims (newer)	Production Ready
EpochAnchor.sol	Oracle-based epoch finalization with Merkle roots	Production Ready
SplitEngine.sol	Governance-controlled split calculator	Production Ready
EligibilityPolicy.sol	On-chain eligibility rules per role	Production Ready
MockUSDT.sol	Test ERC20 token	Test Only

Dashboard (Next.js 16)

[OK] Login with 5 role-based test accounts + role-based route guards

[OK] Admin command center with live KPIs + SSE real-time streaming

[OK] Node operator dashboard with earnings tracking + claim interface

[OK] Builder API key management (create, list, revoke) + usage stats

[OK] Network stats with 3s auto-refresh + feature flags admin panel

[PARTIAL] Forensics, growth, settlement pages (structure exists, minimal UI)

[PARTIAL] Distributor and enterprise dashboards (skeleton only)

[MISSING] Node setup wizard (empty page), wallet connection, marketing pages

SECTION 3 — MISSING PRODUCT LAYERS

For Real Users

- User onboarding flow — No email verification, no KYC, no wallet connection wizard
- Node setup wizard — Page exists but zero implementation
- Payment integration — No fiat on-ramp (MoonPay referenced but not wired)
- User documentation — /docs page is just a link, no searchable docs or Swagger/OpenAPI
- Support system — Ticket management UI exists, no backend ticket handling
- Notifications — Toast UI exists, no push/email notification system

For Real Demand

- Workload marketplace — Job listing structure exists, no public marketplace UI
- API documentation — No Swagger/OpenAPI spec for builder integration
- SDK/Client libraries — No npm package for API consumers
- Usage-based billing — Enterprise billing structure exists, no metered pipeline
- SLA guarantees — Engine exists, no enforceable SLA contracts

For Real Revenue

- EVM settlement — Contract calls stubbed, no actual on-chain transactions from backend
- Dynamic pricing — Hardcoded prices in OP_CONFIG, no demand-based adjustment
- Payment processing — No payment gateway integration (Stripe, MoonPay, etc.)
- Invoice generation — Billing tables exist, no PDF invoice generation
- Financial audit trail — No comprehensive financial audit logging

SECTION 4 — END-TO-END FLOW ANALYSIS

Tracing the complete path: User registration through workload execution to revenue claims.

1. User Registration

```
Flow: User -> POST /auth/register -> SHA256 hash -> INSERT users -> JWT -> Dashboard
```

Gap: No email verification, weak password hashing (SHA256 not bcrypt), no CSRF protection

2. Node Registration

```
Flow: Operator -> POST /node/register -> INSERT registered_nodes -> pair_codes -> Active
```

Gap: Works end-to-end, but NO sync with on-chain NodeRegistryV2 contract

3. Workload Submission

```
Flow: Builder -> POST /v1/jobs -> JobProducer -> Redis Stream -> Consumer -> Node
```

Gap: Job enqueue works. Consumer-to-node assignment incomplete. No execution verification.

4. Revenue Recording

```
Flow: Workload complete -> RevenueOracle -> INSERT revenue_events_v2
```

Gap: Events recorded but many are SIMULATED, not from verified real workloads

5. Epoch Aggregation

```
Flow: EpochScheduler (60s) -> SUM events -> 50/30/20 split -> INSERT epoch_earnings
```

Gap: Works atomically. BUT: operations_engine.js:311 has `db` undefined bug on alt path.

6. Claims & Withdrawal

```
Flow: Operator -> POST /node-api/claim -> Check balance -> Update status
```

Gap: Balance tracking works in DB. On-chain settlement NOT wired. No real USDT movement.

7. Admin Monitoring

```
Flow: Admin -> /admin/command-center -> GET /admin-api/stats -> Real-time KPIs
```

Gap: Works end-to-end. SSE streaming operational.

Critical Breaks in Flow

BREAK: Step 3->4: No verified workload execution = revenue is simulated/synthetic

BREAK: Step 6: No on-chain settlement = withdrawals are database-only, no real USDT movement

BREAK: Step 2: No sync between DB node registration and on-chain NodeRegistryV2

BREAK: Simulated parts: SETTLEMENT_ADAPTER=SIMULATED by default; /dev/generate-activity creates fake data with NO AUTH

SECTION 5 — ARCHITECTURE AUDIT

Service Separation

Monorepo structure: apps/api, apps/dashboard, services/scheduler, packages/, agents/. The backend is a **monolith** — all engines, routes, and schedulers run in a single Express process. No microservice separation, no message bus between components. Frontend is properly separated as a standalone Next.js application.

Coupling Issues

- OperationsEngine is a god object — handles pricing, ops counting, epoch finalization, balance queries
- pg_adapter.js contains SQLite-to-PostgreSQL translation layer (legacy coupling from migration)
- Route files directly import engine instances (tight coupling, no dependency injection)
- Config spread across env.js, schema.js, and inline constants in multiple files

Scaling Limitations

- Single-threaded Node.js — CPU-bound work blocks event loop
- Epoch scheduler uses in-memory state — cannot run multiple instances safely
- No horizontal scaling strategy (no load balancer, no session store, no CDN)
- PostgreSQL pool limited to 20 connections; Redis Streams has no circuit breaker

Single Points of Failure

Component	Risk	Impact
PostgreSQL	Single instance, no replica	Total data loss on failure
Redis	No persistence, no cluster	All queued jobs lost on restart
API Server	Single process	Complete service outage
EpochAnchor Oracle	Single account	Epoch finalization stops if compromised
JWT_SECRET	Already leaked in git history	All tokens can be forged

SECTION 6 — DATABASE AUDIT

60+ tables auto-created via schema.js on startup. No migration framework. PostgreSQL with pg_adapter providing SQLite compatibility translation.

Missing Critical Indexes (Performance Impact)

Table.Column	Query Frequency	Impact
revenue_events_v2.created_at	15+ queries/min	Full table scan every epoch cycle (60s)
revenue_events_v2.client_id	10+ queries	Admin dashboard, ledger queries
revenue_events_v2.node_id	8+ queries	Node performance tracking
revenue_events_v2.epoch_id	6+ queries	Epoch finalization, settlement
registered_nodes.wallet	20+ queries	Node lookup, heartbeat updates
registered_nodes.last_heartbeat	8+ queries	Fleet readiness calculations
epoch_earnings.wallet_or_node_id	5+ queries	Balance queries, claim validation

Consistency Risks

- Foreign keys WITHOUT CASCADE rules -> orphaned records on entity deletion
- No CHECK constraints on numeric fields — amounts can be negative
- Status fields are TEXT with no ENUM constraint — invalid states possible
- No SERIALIZABLE isolation on epoch finalization — double-counting risk under concurrency
- N+1 query pattern in finalizeEpoch() — 200-300 individual INSERTs instead of batch

SQL Injection Findings

! pg_adapter.js:107 — PRAGMA translation uses string interpolation, not parameterized query

! schema.js:12 — hasCol() uses template literal for table name injection

! Impact: Limited by regex \w+ extraction, but violates parameterized query principles

Missing Tables

- audit_log — No financial operation audit trail
- rate_limit_counters — No distributed rate limiting support
- session_store — No server-side session management
- scheduled_tasks — No persistent job scheduling table

SECTION 7 — DOCKER & INFRASTRUCTURE AUDIT

Container Issues

Issue	File	Severity
Runs as root (no USER directive)	apps/api/Dockerfile	HIGH
Uses `npm run dev` in production	infrastructure/docker/Dockerfile.frontend	CRITICAL
Outdated Node 18 base image	services/scheduler/Dockerfile	MEDIUM
Build-time URL baked in, not runtime configurable	apps/dashboard/Dockerfile.web	MEDIUM
Inconsistent base images (slim vs alpine)	Multiple files	LOW

Infrastructure Gaps

- No custom Docker networks — all services on default bridge, no isolation
- No container resource limits (CPU, memory) in docker-compose
- No log rotation configured — unbounded disk usage
- Redis: no ACL, no persistence, no cluster — data lost on restart
- PostgreSQL: no replica, no automated backups, no point-in-time recovery
- No container image registry — images not pushed to DockerHub/GHCR/ECR
- Database credentials hardcoded in docker-compose files

Monitoring Stack

Prometheus, Grafana, and AlertManager are provisioned but largely empty: Prometheus only scrapes the API (no postgres/redis/node exporters), AlertManager has no alert rules defined, and Grafana has no dashboards configured. There is no centralized logging solution (no ELK, Loki, or Splunk).

SECTION 8 — CODEBASE HEALTH

Dead Code / Cleanup Candidates

File Path	Reason	Confidence
SEED_FLEET.js (root)	Dev seed helper, not imported	HIGH
SEED_FULL_DATAv2.js (root)	Dev seed helper, not imported	HIGH
src/nodes/node_onboarding.js.backup	.backup suffix, superseded	HIGH
src/gateway/routes/node_network.js.backup	.backup suffix, superseded	HIGH
src/execution/execution_router_legacy.js	Legacy suffix, replaced	HIGH
Integration.test.js.disabled	.disabled suffix, not running	HIGH
infrastructure/docker/ (entire dir)	Duplicated by infra/docker/	MEDIUM
src/utils/lib/ (9.8MB)	Vendored OZ + forge-std libs	MEDIUM
ClaimsContract.sol	Possibly superseded by ClaimsWithdrawals.sol	LOW

Duplications

- Two Dockerfile directories: infra/docker/ AND infrastructure/docker/
- Two docker-compose configs with overlapping service definitions
- Multiple .env files: .env, .env.docker, .env.sandbox, .env.production, .env.production.local

Debug Artifacts in Production Code

- Hardcoded debug ingest URL in admin_control_room_api.js (http://127.0.0.1:7363)
- console.log("[DEBUG]...") statements scattered across multiple files
- /dev/generate-activity endpoint with NO authentication check
- /dev/token endpoint generates admin_super JWT tokens
- 15+ TODO/FIXME comments in production code

Git Branch Hygiene

93 total branches (local + remote). 10+ archive/* branches, multiple claude/* worktree branches, duplicate names (develop, develop-clean), and no documented branching strategy or CONTRIBUTING.md.

SECTION 9 — SECURITY AUDIT

#	Vulnerability	Severity	Details
1	Secrets committed to git	CRITICAL	JWT_SECRET, DATABASE_URL (with password), API keys in .env file. Recoverable from git history.
2	Weak password hashing	HIGH	SHA256 + static salt instead of bcrypt. No per-password salt, no work factor. Rainbow table vulnerable.
3	Open debug endpoints	HIGH	/dev/token generates admin_super JWT with no rate limit. /dev/generate-activity has no auth. NODE_ENV guard only.
4	Missing CSRF protection	HIGH	No CSRF tokens on any state-changing POST endpoints (login, register, claim, admin actions).
5	CORS defaults to allow all	MEDIUM-HIGH	Empty CORS_ORIGINS accepts all origins. Combined with credentials:true this is dangerous.
6	Admin key timing attack	MEDIUM	Simple string comparison (not crypto.timingSafeEqual). Lockout threshold too high at 10 failures.
7	JWT expiration too long	MEDIUM	Default 24-hour access tokens. Industry standard is 15-30 minutes. No token revocation mechanism.
8	Weak rate limiting	MEDIUM	Auth rate limit 10/min (too high). No account lockout. No distributed rate limiting (memory-based only).
9	Profit guard disabled	MEDIUM	DISABLE_PROFIT_GUARD=true in .env. Financial safety checks completely bypassed.

Smart Contract Security

STRONG: Proper AccessControl, ReentrancyGuard, Pausable on all contracts

STRONG: SafeERC20 for token transfers, Merkle proof verification

STRONG: Recent audit fixes addressed critical access control gaps

RISK: RevenueVault has no max withdrawal limit — trusts Claims contracts entirely

RISK: Single oracle for EpochAnchor — compromise = fraudulent merkle roots

RISK: No upgrade mechanism (immutable contracts) — bugs require redeployment + migration

SECTION 10 — PERFORMANCE & SCALE

Bottlenecks

- PostgreSQL: 20 max connections shared across all operations (heartbeats, queries, writes)
- Epoch aggregation: N+1 query pattern — 200-300 individual INSERTs per 60s cycle
- Missing indexes on revenue_events_v2 — full table scans every epoch cycle
- Single-threaded Node.js — CPU-bound operations (crypto, merkle trees) block event loop
- No pagination on some admin list endpoints — unbounded result sets possible

Failure Scenarios

Scenario	Impact	Recovery
PostgreSQL down	API crashes after 5 retries	Manual restart, no auto-recovery
Redis down	Job queue fails silently	Jobs permanently lost (no persistence)
Memory leak in schedulers	OOM after hours/days	No periodic restart configured
JWT_SECRET rotation	All sessions invalidated	No graceful migration path
100+ node heartbeat storm	Connection pool exhaustion	Service degradation/crash
Epoch double-finalization	Duplicate earnings records	No SERIALIZABLE isolation

SECTION 11 — UI/UX PRODUCT READINESS

Dashboard Usability

[OK] Admin command center — functional with live data and SSE streaming

[OK] Role-based navigation — properly filtered per role

[OK] Dark theme — consistent across all pages (Tailwind + shadcn/ui)

[OK] Mobile responsive — sidebar collapses, mobile nav drawer

[PARTIAL] Loading states — skeleton loaders exist but not on all pages

[PARTIAL] Error handling — error banners exist but some pages fail silently

User Journey Gaps

Role	Can Do	Dead End At
New User	Visit landing pages	Cannot self-register with wallet
Node Operator	See dashboard, view earnings	Cannot complete setup wizard
Builder	Create API keys, view usage	No API documentation to integrate
Enterprise	View dashboard skeleton	Cannot deposit funds or use services
Distributor	See referral tracking	Cannot generate referral links

SECTION 12 — CI/CD & GITHUB READINESS

Workflow Status

Workflow	Purpose	Status
satelink-ci.yml	Main CI (contracts, backend, frontend)	Partial — works but no coverage
deploy.yml	Deployment trigger	Broken — no registry push, no health checks
e2e.yml	E2E tests with postgres/redis	Partial — needs more test coverage
contracts-deploy.yml	Foundry contract deployment	Partial — no mainnet approval gate
ci.yml	Duplicate CI file	BROKEN — duplicate `name:` field, fails parsing

Missing CI/CD Capabilities

- No container image registry push (no DockerHub/GHCR/ECR)
- No health check validation post-deployment
- No rollback mechanism on failed deploys
- No deployment approval gates for production
- No staging -> production promotion pipeline
- No npm audit or Trivy security scanning in pipeline
- No code coverage thresholds enforced
- Frontend has passWithNoTests flag — tests can be silently skipped

SECTION 13 — LAUNCH READINESS SCORE

Scale: 0 = nothing built, 10 = production ready

Category	Score	Justification
Backend Core	6 / 10	Auth, epochs, admin routes work. Settlement and workloads partial. Critical bugs.
Smart Contracts	8 / 10	All 9 contracts compile, tested, audited. Needs multi-sig oracle and proxy pattern.
Frontend / Dashboard	5 / 10	Admin portal functional. User-facing flows missing. Dev login only.
Database	5 / 10	60+ tables, good schema design. Missing indexes, no migrations, consistency risks.
Infrastructure	3 / 10	Docker works for dev. No prod hardening, no monitoring, no backups.
Security	2 / 10	Secrets in git, SHA256 passwords, open debug endpoints, no CSRF.
CI/CD	3 / 10	Workflows exist but incomplete. No image registry, no rollback, broken ci.yml.
Product Completeness	3 / 10	Admin tools work. User onboarding, payments, settlement missing.
Documentation	2 / 10	No API docs, no deployment runbooks, no architecture docs.
Observability	2 / 10	Prometheus basic, zero alerts, zero dashboards, no centralized logging.

OVERALL LAUNCH READINESS: 3.9 / 10

VERDICT: The system has a solid **foundation** — the backend architecture, smart contracts, and admin tooling demonstrate real engineering depth. However, it is **NOT ready** for production launch with real users and real money. Critical security vulnerabilities (secrets in git, weak password hashing), missing product flows (no wallet connect, no settlement wiring), and infrastructure gaps (no backups, no monitoring alerts) must be addressed before any launch.

SECTION 14 — CLEANUP PLAN (SAFE MODE)

DO NOT DELETE — Review and verify each candidate before action. These are files that *appear* unused based on code analysis.

File Path	Reason	Confidence
/SEED_FLEET.js	Dev seed helper in root, not imported	HIGH
/SEED_FULL_DATAv2.js	Dev seed helper in root, not imported	HIGH
src/nodes/node_onboarding.js.backup	.backup suffix, superseded	HIGH
src/gateway/routes/node_network.js.backup	.backup suffix, superseded	HIGH
src/execution/execution_router_legacy.js	Legacy suffix, likely replaced	HIGH
tests/unit/Integration.test.js.disabled	.disabled suffix, not running	HIGH
infrastructure/docker/ (entire dir)	Duplicated by infra/docker/	MEDIUM
src/utils/lib/ (9.8MB vendored)	Should use npm/forge install	MEDIUM
docker-compose.dev.yml (root)	May be superseded by infra/	MEDIUM
ClaimsContract.sol	Possibly superseded by ClaimsWithdrawals	LOW

Git Branch Cleanup (93 -> ~10 recommended)

- archive/* (10+ branches) — Archive tags or delete
- claude/* worktree branches — Clean up after verifying merged state
- develop-clean — Unclear purpose, may duplicate develop
- debug/node-register-investigation — Debug branch, issue resolved on main
- Multiple fix/* branches — Verify merged status before pruning

SECTION 15 — EXECUTION ROADMAP

PHASE 1 — CRITICAL BLOCKERS (Weeks 1-2)

Priority: Security fixes + bugs that prevent core functionality

1.1 Secret Rotation & Scrub

- [ ] Rotate ALL secrets (JWT_SECRET, API keys, DB passwords) immediately
- [ ] Run bfg-repo-cleaner to scrub .env from git history
- [ ] Implement .env validation on startup (fail-fast if critical vars missing)
- [ ] Set up secret management (AWS Secrets Manager or HashiCorp Vault)

1.2 Fix Critical Bugs

- [ ] Fix operations_engine.js:311 — change `db` to `this.db`
- [ ] Fix futures_escrow.js — add missing `await` keywords (lines 14, 40, 97)
- [ ] Fix job_escrow.js:25 — add missing `await`

1.3 Security Hardening

- [ ] Replace SHA256 password hashing with bcrypt (12 rounds)
- [ ] Add CSRF protection to all state-changing POST endpoints
- [ ] Fix CORS to require explicit origins in production (fail if CORS_ORIGINS empty)
- [ ] Use crypto.timingSafeEqual for admin API key comparison
- [ ] Reduce JWT expiration to 15 minutes + implement refresh token flow
- [ ] Remove/gate all /dev/* endpoints behind strict NODE_ENV === 'development' checks
- [ ] Add rate limiting: 5 auth attempts/minute, account lockout after 3 failures

1.4 Database Indexes

- [ ] Add indexes on all hot columns (revenue_events_v2, registered_nodes, epoch_earnings)
- [ ] Add CHECK constraints on financial amount fields (amount >= 0)

PHASE 2 — PRODUCT COMPLETION (Weeks 3-6)

Priority: Core user flows that enable real usage

2.1 User Onboarding

- [ ] Implement wallet connection (MetaMask, WalletConnect)
- [ ] Add email verification flow for auth/register
- [ ] Build node setup wizard (currently empty /node/setup page)
- [ ] Create builder API documentation (Swagger/OpenAPI spec)

2.2 Settlement Wiring

[ ] Connect backend to smart contracts via ethers.js

[ ] Wire ClaimsWithdrawals.sol to claim/withdraw API endpoints

[ ] Implement Merkle proof generation matching contract encoding exactly

[ ] Wire RevenueDistributor.sol to epoch finalization

[ ] Deploy all contracts to Fuse Spark testnet

2.3 Payment Pipeline

[ ] Wire enterprise deposit flow to RevenueVault on-chain

[ ] Implement usage-based metering for builder API calls

[ ] Build invoice generation + financial audit logging table

2.4 Workload Pipeline

[ ] Complete Redis Streams consumer -> node assignment flow

[ ] Wire RPC gateway to actual node clusters

[ ] Implement workload verification (prove node executed work)

[ ] Connect revenue events to verified workload completions only

PHASE 3 — PRODUCTION HARDENING (Weeks 7-10)

Priority: Infrastructure and reliability for real traffic

3.1 Infrastructure

[ ] Standardize all Dockerfiles on node:20-alpine with non-root USER

[ ] Add resource limits to docker-compose (CPU, memory)

[ ] Set up PostgreSQL replica + automated backups (pg_dump cron)

[ ] Enable Redis persistence (AOF) and configure ACLs

[ ] Configure log rotation for all containers

3.2 CI/CD Pipeline

[ ] Fix broken ci.yml (duplicate name field)

[ ] Add Docker image build + push to GHCR

[ ] Add health check validation + rollback mechanism

[ ] Add npm audit + Trivy image scanning to pipeline

[ ] Set up staging -> production promotion with approval gate

3.3 Observability

[ ] Add Prometheus exporters (postgres, redis, node)

[ ] Create Grafana dashboards (latency, errors, queue depth, epochs)

[ ] Define AlertManager rules (error rate, epoch failures, vault balance)

[ ] Set up centralized logging (Loki or ELK stack)

3.4 Database Hardening

- [ ] Implement migration framework (node-pg-migrate)
- [ ] Add ON DELETE CASCADE/SET NULL to foreign keys
- [ ] Convert N+1 queries to batch operations
- [ ] Add SERIALIZABLE isolation for epoch finalization

3.5 Contract Hardening

- [ ] Deploy multi-sig for ORACLE_ROLE and ADMIN roles
- [ ] Set up separate pool wallets for RevenueDistributor
- [ ] Consider UUPS proxy for RevenueVault (upgradability)
- [ ] Set up vault balance monitoring (alert if claims > deposits)

PHASE 4 — LAUNCH (Weeks 11-14)

Priority: Go-live preparation and verification

4.1 Testing

- [ ] E2E flow test: register -> deposit -> workload -> revenue -> claim -> withdraw
- [ ] Load test: 100+ concurrent nodes, 1000+ ops/minute
- [ ] Third-party smart contract security audit
- [ ] Penetration test on API and frontend

4.2 Deploy & Verify

- [ ] Deploy contracts to Fuse mainnet + verify on explorer
- [ ] Deploy backend + frontend to production infrastructure
- [ ] Configure production monitoring and alerts
- [ ] Verify all contract roles and grants

4.3 Launch Checklist

- [ ] All secrets rotated and stored in secret manager
- [ ] All .env files scrubbed from git history
- [ ] bcrypt password hashing active
- [ ] CSRF + CORS locked to production domains
- [ ] Debug endpoints removed/disabled
- [ ] Database indexes + backups verified
- [ ] Monitoring dashboards + alert rules live
- [ ] Smart contracts verified + multi-sig operational
- [ ] Incident response runbook documented
- [ ] On-call rotation established

Timeline Summary

Phase	Focus	Duration	Target
Phase 1	Critical Blockers (Security + Bugs)	Weeks 1-2	April 2026
Phase 2	Product Completion (User Flows + Settlement)	Weeks 3-6	May 2026
Phase 3	Production Hardening (Infra + CI/CD + Observability)	Weeks 7-10	June 2026
Phase 4	Launch (Testing + Deploy + Go-Live)	Weeks 11-14	July 2026

Estimated Launch Date: July 2026 (~14 weeks from audit date)